

Felipe Mesa · Diego Rodriguez
202123007 | 202225217

VeriLang

Lenguaje de especificacion formal implementado en **Rascal MPL**
Analizador, verificador de tipos y validador semantico para estructuras algebraicas



VeriLang permite definir estructuras algebraicas compuestas por espacios, operadores, variables, reglas de reescritura y expresiones logicas. Los programas se escriben en archivos `.vl` y son procesados por la cadena de herramientas VeriLang, que los parsea, verifica tipos y valida semanticamente.

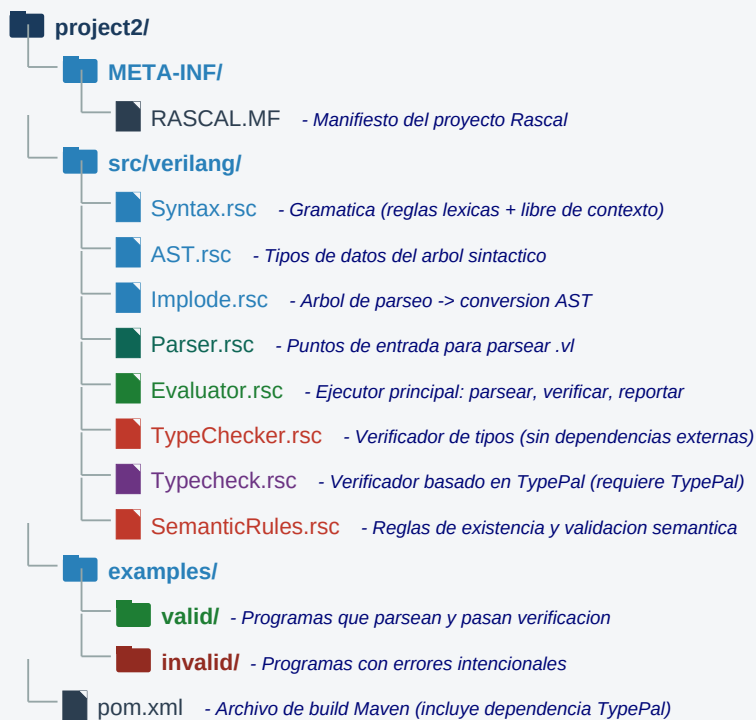
1. Descripcion General

VeriLang es un lenguaje de especificacion formal implementado en Rascal. Permite a los usuarios definir estructuras algebraicas compuestas de espacios (sorts), operadores, variables, reglas de reescritura y expresiones logicas.

Autor	Codigo
Felipe Mesa	202123007
Diego Rodriguez	202225217

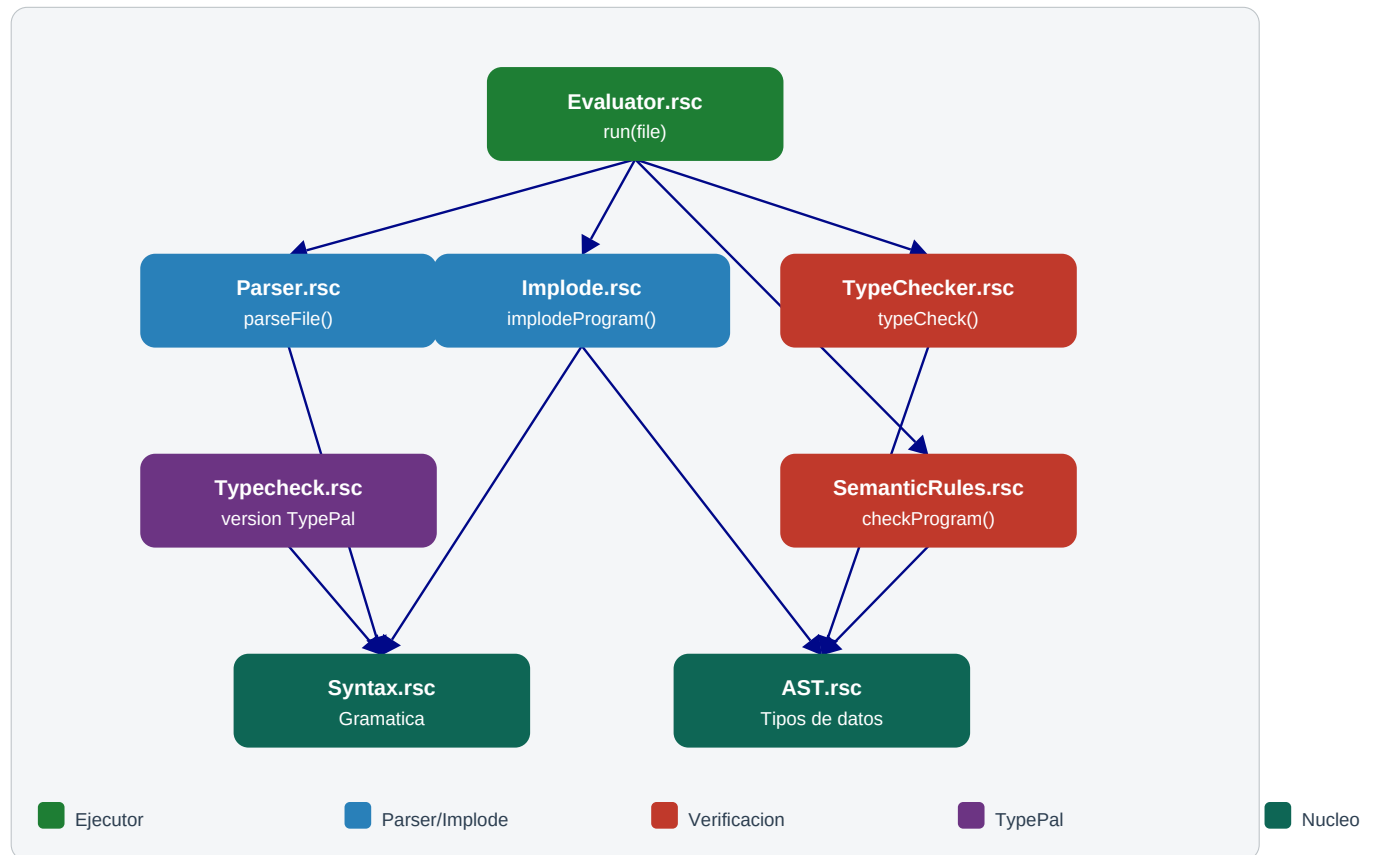
2. Estructura del Proyecto

Organizacion modular del proyecto: cada archivo .rsc tiene una responsabilidad unica dentro del pipeline de compilacion.



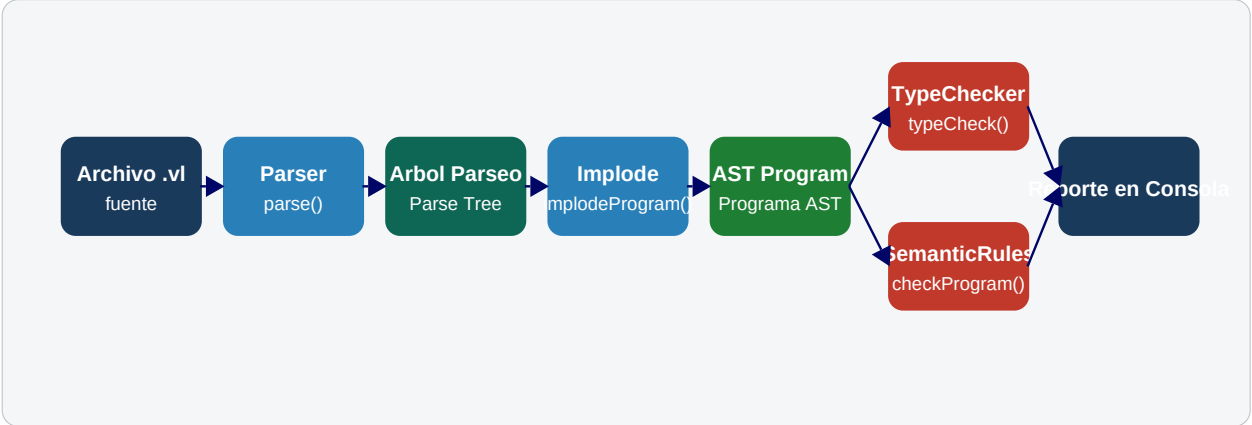
3. Arquitectura de Modulos

El modulo `Evaluator.rsc` es el punto de entrada principal. Orquesta el parsing, la conversion a AST, la verificacion de tipos y la validacion semantica. Los modulos de nucleo (`Syntax.rsc`, `AST.rsc`) son compartidos por multiples ramas.



4. Pipeline de Procesamiento

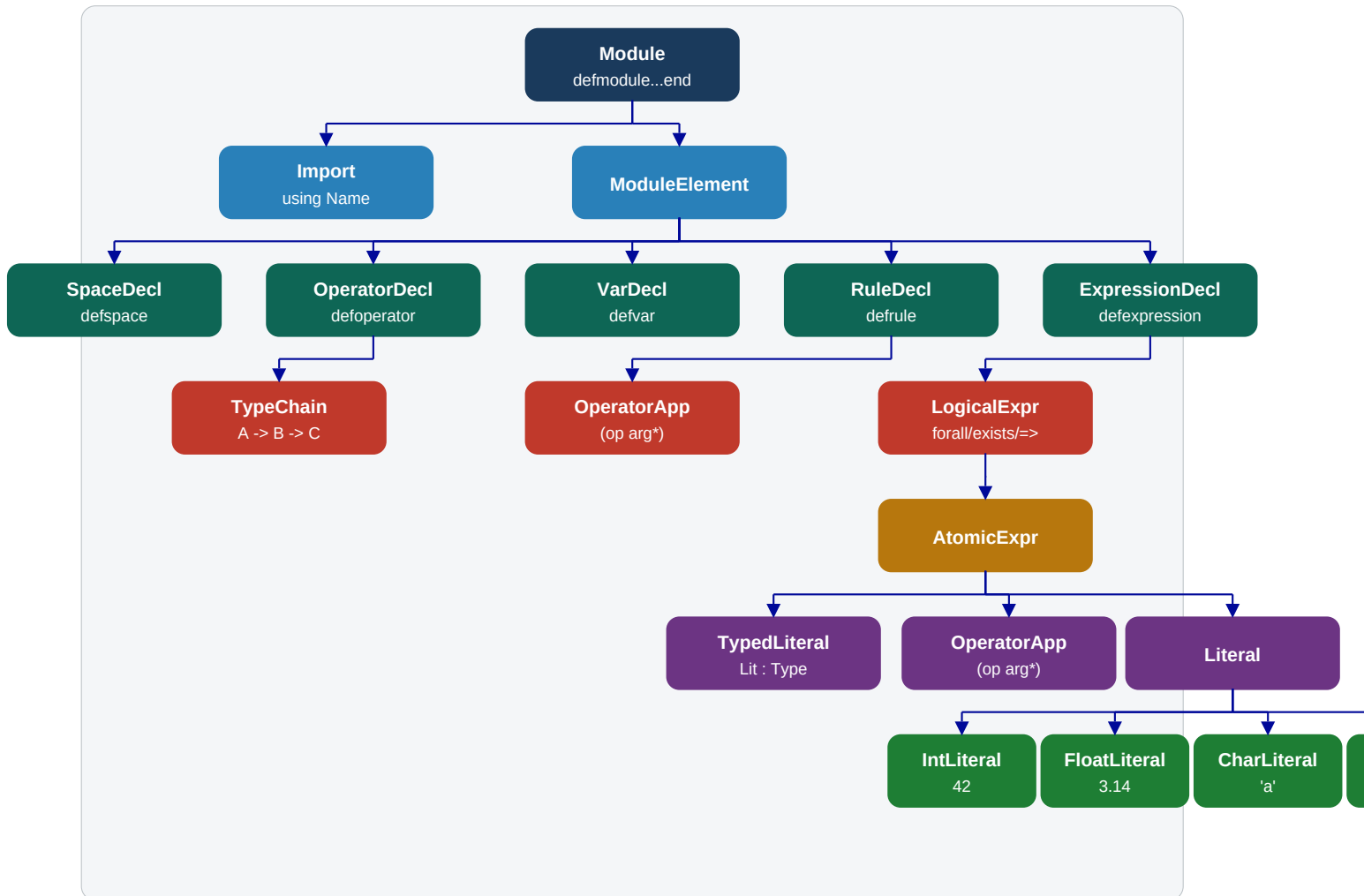
El pipeline transforma un archivo fuente `.vl` en un reporte de consola en cinco etapas. Las fases de verificacion de tipos y validacion semantica se ejecutan en paralelo sobre el mismo AST.



Parseo	Parser.rsc lee el archivo fuente y produce un arbol de parseo concreto usando la gramatica definida en Syntax.rsc.
Implode	Implode.rsc convierte el arbol de parseo concreto en un AST tipado mediante pattern matching exhaustivo.
Verificacion de tipos	TypeChecker.rsc recorre el AST y verifica que todas las anotaciones de tipo sean correctas.
Validacion semantica	SemanticRules.rsc aplica 8 reglas de existencia: operadores, tipos, variables, espacios y literales.
Reporte	Evaluator.rsc consolida los resultados de ambas fases y los imprime en consola de forma estructurada.

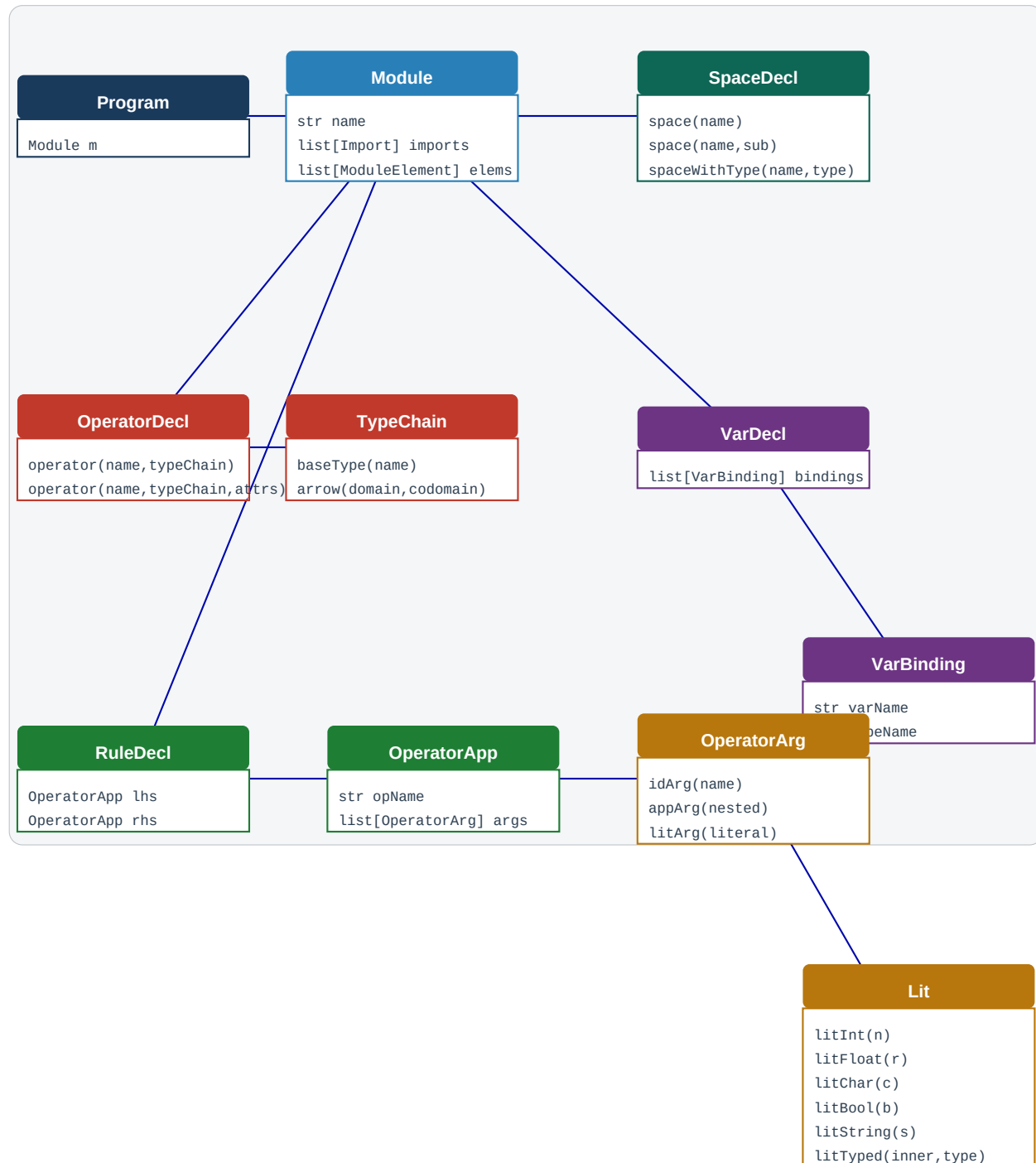
5. Descripcion de la Gramatica

El siguiente diagrama muestra la jerarquia de produccion de la gramatica de VeriLang, desde el nodo raiz **Module** hasta los literales hoja.



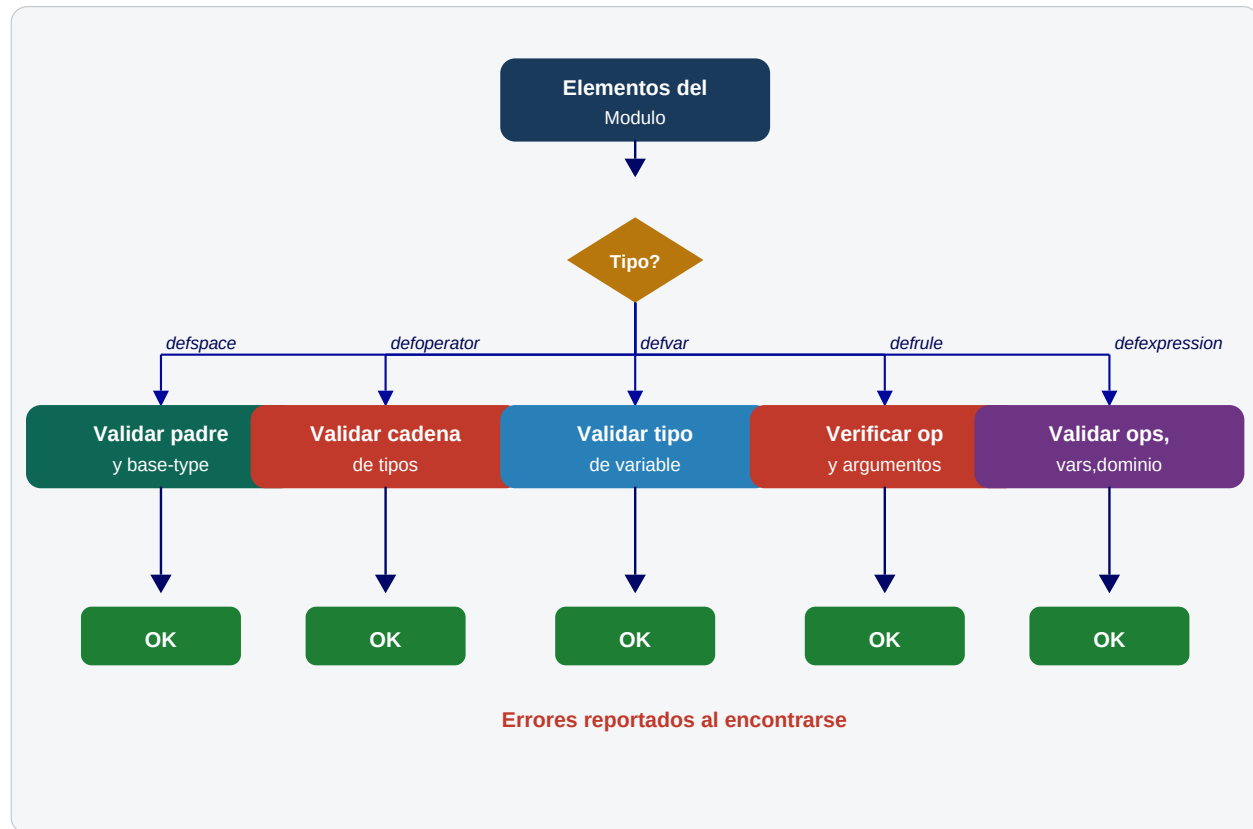
6. Tipos de Datos AST

Cada nodo del AST es un tipo de dato algebraico en Rascal. El diagrama de clases siguiente muestra las relaciones entre los principales nodos.



7. Flujo de Verificacion de Tipos

El verificador de tipos recorre todos los elementos del modulo y aplica reglas especificas segun el tipo de declaracion.



8. Referencia del Lenguaje

Modulo

```
defmodule Name
  using OtherModule
  ... declaraciones ...
end
```

Declaracion de espacio

```
defspace Name end
defspace Name < ParentSpace end
defspace Name : BaseType end // nueva en esta version
```

La tercera forma anota el espacio con un tipo primitivo base: Int, Bool, Char, String, Float.

Declaracion de operador

```
defoperator name : TipoA -> TipoB -> TipoRetorno end
defoperator name : TipoA -> TipoRetorno [ atributo ] end
```

Declaracion de variable

```
defvar x : NombreEspacio , y : NombreEspacio end
```

Regla de reescritura

```
defrule (opLHI arg1 arg2) -> (opLHD arg1) end
```

Expresion

```
defexpression forall x in Espacio . (op x) end
defexpression exists x in Espacio . (op x) => (id x) end
```

Literales y anotaciones de tipo

Tipo	Ejemplo	Con anotacion
Int	42	42:Int
Float	3.14	3.14:Float
Char	'a'	'a':Char
Bool	true, false	true:Bool
String	"hola"	"hola":String

Operadores logicos (precedencia de menor a mayor)

```
forall x in S . Expr
exists x in S . Expr
Expr equiv Expr
Expr => Expr
Expr and Expr
Expr or Expr
neg Expr
( Expr )
```


Tipos primitivos incorporados

Los siguientes nombres de tipo son siempre validos sin necesidad de declaracion de `space`:

- `Int`
- `Bool`
- `Char`
- `String`
- `Float`

9. Reglas Semanticas (Tarea 6)

El modulo SemanticRules verifica que cada nombre usado dentro de una construccion este declarado. Se aplican 8 reglas:

Regla	Descripcion
1. Aplicaciones de operador	El operador y cada argumento identificador deben estar declarados.
2. Cadenas de tipo	Cada nombre de espacio en la firma de un operador debe ser conocido.
3. Declaraciones de variable	El tipo declarado debe ser un espacio conocido.
4. Relaciones de sub-espacio	El espacio padre debe estar declarado o ser primitivo.
5. Espacios tipados	La anotacion de tipo base debe ser un tipo conocido.
6. Cuantificadores	El dominio debe ser un espacio declarado o primitivo.
7. Expresiones de pertenencia	Tanto el elemento como la coleccion deben estar declarados.
8. Literales tipados	El tipo de anotacion debe ser un espacio conocido o primitivo.

10. Programas de Ejemplo

Valido: modulo Bool (examples/valid/02-bool.vl)

```
defmodule Bool
defspace Bool end
defoperator not : Bool -> Bool end
defoperator and : Bool -> Bool -> Bool [ associative commutative ] end
defoperator or : Bool -> Bool -> Bool [ associative commutative ] end
defvar p : Bool , q : Bool end
defrule (not (not p)) -> (id p) end
defexpression forall p in Bool . (not p) end
end
```

Valido: modulo Nat con espacios tipados (examples/valid/04-typed.vl)

```
defmodule TypedExample
defspace IntSpace : Int end
defspace BoolSpace : Bool end
defspace Nat < IntSpace end
defoperator zero : IntSpace end
defoperator succ : IntSpace -> IntSpace end
defoperator add : IntSpace -> IntSpace -> IntSpace [ associative id : zero ] end
defvar n : IntSpace , m : IntSpace end
defrule (add (succ n) m) -> (succ (add n m)) end
defexpression forall n in IntSpace . (add zero n) => (id n) end
end
```

Invalido: operador no declarado (examples/invalid/05-bad-element.vl)

```
defmodule BadElement
defspace Nat end
defoperator succ : Nat -> Nat end
defvar n : Nat end
defrule (ghost n) -> (succ n) end // "ghost" no esta declarado
end
```

Error esperado	La regla semantica 1 detecta que "ghost" no esta declarado como operador y reporta el error.
-----------------------	--

11. Como Ejecutar

1. Abrir el terminal Rascal en VS Code

Abrir la Paleta de Comandos (Ctrl+Shift+P) y ejecutar **Rascal: Start Terminal**.

2. Ejecutar un programa .vl

```
import verilang::Evaluator;  
run(|file:///C:/ruta/al/archivo.vl|);
```

3. Ejecutar solo el parser

```
import verilang::Parser;  
parseFile(|file:///C:/ruta/al/archivo.vl|);
```

4. Ejecutar solo el verificador de tipos

```
import verilang::TypeChecker;  
checkAndReport(|file:///C:/ruta/al/archivo.vl|);
```

5. Ejecutar el verificador basado en TypePal (requiere TypePal via Maven)

```
import verilang::Typecheck;  
checkAndReport(|file:///C:/ruta/al/archivo.vl|);
```

6. Ejecutar los ejemplos proporcionados

```
import verilang::Evaluator;  
run(|file:///C:/.../examples/valid/02-bool.vl|);  
run(|file:///C:/.../examples/valid/03-nat.vl|);  
run(|file:///C:/.../examples/valid/04-typed.vl|);  
run(|file:///C:/.../examples/invalid/04-bad-type.vl|);  
run(|file:///C:/.../examples/invalid/05-bad-element.vl|);
```

12. Instalacion de TypePal

TypePal esta declarado en pom.xml como dependencia Maven. Para descargarlo y activarlo:

- En VS Code, abrir la Paleta de Comandos y ejecutar **Java: Reload Projects** (o hacer clic derecho en pom.xml y seleccionar **Reload project**).
- Reiniciar el terminal Rascal.
- Importar verilang::Typecheck para usar el verificador basado en TypePal.

Requisitos

Requisito	Version/Detalle
Visual Studio Code	Con la extension Rascal instalada
Java	11 o superior
Maven	Para resolucion de la dependencia TypePal